

Baufauftrag — „ Gründlich“ wird ein echter Umschalter mit Haken

Von: Claudia (Bot-Sitzung auf dem VPS, nur Leserecht am Bot-Repo) **Für:** Mick (Migrationssitzung am Mac) — dort liegt das Schreibrecht **Datum:** Montag, 27.07.2026
Anlass: Adams Sprachnachricht vom 27.07., 11:35 Uhr **Codestand, gegen den geprüft wurde:** `bot.py` auf dem VPS, gelesen am 27.07. gegen 11:50 Uhr. Alle Zeilennummern beziehen sich auf diesen Stand und sind vor dem Eingriff gegenzuprüfen.

1. Was Adam will (seine Worte)

„Wir müssen noch was einbauen in den Gründlich-Taster, dass man da sieht, wenn es angeschaltet ist, ist ein Haken dran. Wenn nicht, ist er weg. ... Ist so lange praktisch, wie ich nicht aus Versehen auf den Knopf komme, wie jetzt, weil ich kann ihn nicht mehr ausschalten. ... Deswegen mach den doch einfach zum An- und Ausschalten. Das ist eh in Ordnung, weil es kann ja sein, dass ich intensivere Sachen hintereinander recherchiert haben will, dann passt das auch.“

Zwei Forderungen: **sichtbarer Zustand** (Haken) und **abschaltbar** (Umschalter statt Einmal-Aktion). Der Dauerbetrieb ist ausdrücklich gewollt, nicht nur geduldet.

2. Ist-Zustand (im Code belegt, nicht vermutet)

Ort	Stand
<code>bot.py:308</code>	<code>_BTN_THOROUGH = " Gründlich"</code> — nur eine Beschriftung, kein Aktiv-Zustand
<code>bot.py:309</code>	<code>_THOROUGH_PENDING: set[int]</code> — reiner Arbeitsspeicher, überlebt keinen Neustart
<code>bot.py:343-354</code>	<code>_ALL_KEYBOARD_BTNS</code> enthält <code>_BTN_THOROUGH</code>
<code>bot.py:547-578</code>	<code>_main_keyboard(tts_on, model, effort)</code> — Modell und Tiefe kennen Aktiv-Beschriftungen (<code>*_ACTIVE</code>), Gründlich nicht
<code>bot.py:5930-5944</code>	Druck $\Rightarrow$ <code>_THOROUGH_PENDING.add(user_id)</code> . Ein zweiter Druck fügt denselben Eintrag erneut hinzu — es gibt keinen Weg hinaus.
<code>bot.py:5587-5588</code>	

Ort	Stand
	Beim Annehmen einer Nachricht: <code>thorough = user_id in _THOROUGH_PENDING</code> und sofort <code>discard</code> ⇒ verbraucht sich
<code>bot.py:1225-1232</code>	Bei <code>job.thorough</code> : <code>ensure_session(user_id,</code> <code>effort_override="max", fresh=True)</code> ⇒ erzwingt für jede solche Anfrage eine frische Sitzung
<code>bot.py:1420-1426</code>	Nach dem Lauf: <code>close_session(user_id)</code> ⇒ Sitzung wird wieder weggeworfen
<code>bot.py:2370-2371</code>	<code>/status:</code> „ Gründlich ist für die nächste Anfrage vorgemerkt“
<code>bot.py:5040</code>	Selbsttest prüft, dass <code>_THOROUGH_PENDING</code> in <code>cmd_status</code> vorkommt
<code>bot.py:2675-2681</code>	<code>/hilfe</code> beschreibt den Knopf als „nächste Frage besonders sorgfältig“
<code>bot.py:5628-5640</code>	Die Job-Eigenschaft <code>thorough</code> wird in der Nachrichten-Persistenz mitgeschrieben

Damit ist Adams Beobachtung bestätigt: Der Modus lässt sich nur dadurch beenden, dass man irgendeine Nachricht schickt — die ihn verbraucht.

3. Soll-Zustand

1. Der Knopf trägt bei aktivem Modus einen Haken: `Gründlich ✓`.
2. Ein Druck schaltet um — an, aus, an. Beide Richtungen bekommen eine eigene Bestätigung.
3. Der Zustand **überlebt einen Neustart** (er liegt bei Modell und Tiefe).
4. Der Modus gilt für **jede** Nachricht, solange er an ist — ohne dass die Sitzung je Anfrage neu aufgebaut und danach weggeworfen wird.

4. Die Änderungen im Einzelnen

A — Zweite Beschriftung anlegen

Bei `bot.py:308` ergänzen:

```
_BTN_THOROUGH_ACTIVE = " Gründlich ✓"
```

und in `_ALL_KEYBOARD_BTNS` (`bot.py:354`) mit aufnehmen. **Ohne diesen zweiten Schritt wird ein Druck auf den Haken-Knopf nicht als Taste erkannt, sondern als gewöhnliche Frage an den Agenten weitergereicht** — der Fehler ist am 23.07. mit

dem Transkriptions-Knopf schon einmal live passiert und ist im Code an dieser Stelle ausdrücklich vermerkt.

B — Zustand dauerhaft ablegen, `_THOROUGH_PENDING` auflösen

Das Set ersatzlos streichen und durch zwei kleine Helfer ersetzen, die auf demselben Weg arbeiten wie Modell und Tiefe (`_USER_PREFS` + `_save_prefs`):

```
def _thorough_on(user_id: int | None) -> bool:
    if user_id is None:
        return False
    return bool(_USER_PREFS.get(str(user_id), {}).get("thorough"))

def _set_thorough(user_id: int, on: bool) -> None:
    prefs = _USER_PREFS.setdefault(str(user_id), {})
    if on:
        prefs["thorough"] = True
    else:
        prefs.pop("thorough", None)
    _save_prefs(_USER_PREFS)
```

Zwei parallele Wahrheiten (Set **und** Datei) wären eine Fehlerquelle für sich — deshalb fällt das Set weg statt gespiegelt zu werden.

C — Die Tiefe an einer einzigen Stelle erzwingen

In `ensure_session` (`bot.py:2179`) unmittelbar nach der bestehenden `effort`-Zeile:

```
if _thorough_on(user_id) and effort_override is _UNSET:
    effort = "max"
```

Warum dort und nicht beim Aufrufer: Die Sitzung wird an mehreren Stellen neu aufgebaut — beim Modellwechsel (`bot.py:5986`), beim Tiefenwechsel (`bot.py:6032`), nach einem Neustart. Läge die Regel nur im Auftragslauf, verlöre ein Modellwechsel bei aktivem Gründlich still die Tiefe: Der Knopf zeigt weiter den Haken, gearbeitet wird flach. Genau die Sorte Fehler, die niemandem auffällt.

D — Auftragslauf entrümpeln

- `bot.py:1225-1232`: Die Sonderbehandlung entfällt vollständig, es bleibt `sess = await ensure_session(user_id)`. Der Textzusatz `_THOROUGH_PREFIX` (`bot.py:1274`) **bleibt unverändert** — er ist der eigentliche Quellencheck-Auftrag.
- `bot.py:1420-1426`: Beide `close_session`-Aufrufe im Gründlich-Zweig **ersatzlos streichen**.

Das ist der wichtigste Punkt des ganzen Auftrags. Blicke das Schließen stehen, hätte im Dauerbetrieb jede Nachricht eine leere Sitzung: kein Gesprächsfaden, kein Bezug auf die vorige Antwort. Das meldet niemand als Fehler — es sieht aus wie Vergesslichkeit.

E — Knopfdruck: umschalten statt vormerken

`bot.py:5930-5944` ersetzen. Fachlich genau das Muster des Tiefen-Knopfes (`bot.py:6000-6039`), damit sich beide gleich anfühlen:

1. Bedingung auf **beide** Beschriftungen prüfen: `if text in (_BTN_THOROUGH, _BTN_THOROUGH_ACTIVE):`
2. Neuen Zustand bilden: `neu = not _thorough_on(user_id)`, dann `_set_thorough(user_id, neu)`.
3. **Läuft gerade ein Auftrag** (`mb.current_job is not None`): `mb.switch_pending = True` setzen und mit „gilt ab der nächsten Aufgabe“ bestätigen — die laufende Arbeit wird nicht abgebrochen, wie beim Modell- und Tiefenwechsel auch.
4. **Sonst:** `await close_session(user_id)` und `await ensure_session(user_id)`, damit die neue Tiefe sofort greift.
5. Bestätigungstext je Richtung:
 - an: „ Gründlich ist **an** — bis du ihn wieder ausschaltest: aktives Modell, höchste Denktiefe, Pflicht-Quellencheck. Er kostet spürbar mehr Zeit und Kontingent.“
 - aus: „ Gründlich ist **aus** — wieder normales Tempo.“
6. Tastatur mit der neuen Beschriftung mitschicken.

F — Verbrauch beim Annehmen der Nachricht

`bot.py:5587-5588` wird zu:

```
thorough = _thorough_on(user_id)
```

Das `discard` fällt weg. Die Persistenz-Zeile `"thorough": thorough` (`bot.py:5637`) bleibt **unverändert richtig**: Sie hält fest, wie dieser eine Auftrag laufen sollte — wird er nach einem Neustart nachgeholt, behält er seine Gründlichkeit, auch wenn Adam den Modus inzwischen ausgeschaltet hat. Das ist gewollt.

G — Tastatur zeichnet den Zustand

`_main_keyboard` (`bot.py:547`) um einen Parameter erweitern:

```
def _main_keyboard(tts_on: bool, model: str, effort: str | None = None,
                  user_id: int | None = None) -> ReplyKeyboardMarkup:
    thorough = _thorough_on(user_id)
    ...
    if thorough:          # ehrlicher Haken: bei Gründlich läuft alles auf Max
        effort = "max"
```

Der Gründlich-Knopf trägt dann `_BTN_THOROUGH_ACTIVE if thorough else _BTN_THOROUGH` (`bot.py:571` und `573`).

Die Zeile `effort = "max"` ist kein Schönheitsfehler, sondern Notwendigkeit: Ohne sie zeigt die Tastatur weiter „Normal“ mit Haken, während in Wahrheit auf höchster Stufe gearbeitet wird. Ein Haken, der lügt, ist schlimmer als keiner.

H — Alle Aufrufstellen mit `user_id=` versorgen

`_main_keyboard` wird an **15** Stellen in `bot.py` gerufen: 2268, 2338, 2695, 4822, 5448, 5942, 5952, 5960, 5978, 5988, 6005, 6023, 6034, 6050, 6122, 7645. Jede bekommt `user_id=...` mit. Wo eine Sitzung vorliegt, ist es `sess.user_id` beziehungsweise die im Umfeld bereits vorhandene `user_id`.

Ausnahme: Zeile 4822 gehört zum Selbsttest und darf ohne bleiben (siehe J). Die beiden Aufrufe in `scripts/check_hilfe_buttons.py` (Zeilen 80 und 84) bleiben ebenfalls unverändert — dort wird nur die Knopfmenge geprüft.

Eine vergessene Stelle führt dazu, dass der Haken bei bestimmten Bot-Antworten verschwindet, obwohl der Modus an ist. Deshalb der Prüfer in J.

I — Nutzersichtbare Texte nachziehen

- `/status` (`bot.py:2370-2371`): Bedingung auf `_thorough_on(user_id)` umstellen, Text auf „ Gründlich ist **an** (höchste Tiefe · Quellencheck)“.
- `/hilfe` (`bot.py:2680-2681`): „ Gründlich ✓ — Umschalter: solange an, läuft jede Frage mit höchster Tiefe und Pflicht-Quellencheck“. Die Knopf-**Anzahl** in `/hilfe` („(9)“) bleibt gleich, es kommt kein Knopf hinzu.

J — Prüfer nachziehen (sonst ist der Auftrag nicht fertig)

1. **Selbsttest** (`bot.py:5040`): Die Zusicherung auf `_THOROUGH_PENDING` **muss** auf `_thorough_on` umgeschrieben werden, sonst schlägt der Test nach dem Umbau fehl.
2. **Tastatur-Vollständigkeit** (`bot.py:4812-4833`): Die Prüfschleife variiert Modell, Tiefe und Transkriptions-Modus — den Gründlich-Zustand **nicht**. Sie würde die neue Aktiv-Beschriftung also nie zu Gesicht bekommen. Die Schleife um `for thorough in (False, True)`: erweitern und `_USER_PREFS` für die Testkennung entsprechend setzen (im `finally` zurücksetzen, wie es dort schon mit `_STT_MODELS` geschieht).
3. **Neuer Prüfer gegen die vergessene Aufrufstelle** — als eigener Punkt in der Selbsttest-Reihe:

```
def _c_keyboard_userid() -> None:
    """Jeder _main_keyboard-Aufruf muss user_id mitgeben, sonst verschwindet
    der Gründlich-Haken an dieser Stelle still."""
    import re
    quelle = Path(__file__).read_text(encoding="utf-8")
    treffer = [z for z in re.findall(r"_main_keyboard\((?:[^\()|\([^\)]*\))*\)",
    quelle)
                if "user_id" not in z and "def _main_keyboard" not in z]
    # Die Testschleife selbst darf ohne user_id bleiben.
    treffer = [z for z in treffer if "for row in" not in z]
    assert not treffer, f"_main_keyboard ohne user_id: {treffer}"
```

5. Was kann brechen — und wer merkt es

Bruchstelle	Wirkung	Wer merkt es
<code>_BTN_THOROUGH_ACTIVE</code> fehlt in <code>_ALL_KEYBOARD_BTNS</code>	Druck auf den Haken-Knopf landet als Frage beim Agenten	Sofort sichtbar — es kommt eine Antwort auf „Gründlich ✓“. Zusätzlich Selbsttest /Tastatur-Vollständigkeit, sobald J.2 gebaut ist
<code>close_session</code> in D bleibt stehen	Jede Nachricht startet ohne Gesprächsfaden	Niemand — sieht aus wie Vergesslichkeit. Deshalb ist D der Kernpunkt; Abnahmeschritt 4 prüft ihn ausdrücklich
Eine der 15 Aufrufstellen ohne <code>user_id</code>	Haken verschwindet nach bestimmten Bot-Antworten	Prüfer J.3
Zustand nicht in <code>_USER_PREFS</code>	Haken nach jedem Neustart weg, Adam merkt es an einer flachen Antwort	Abnahmeschritt 3; im Betrieb der / <code>status</code> -Eintrag
Tiefe nur im Auftragslauf gesetzt statt in <code>ensure_session</code>	Nach einem Modellwechsel läuft es flach, Haken bleibt stehen	Niemand — deshalb steht die Regel in C an der einen zentralen Stelle
Tiefen-Knopf wirkt bei aktivem Gründlich nicht mehr	Adam drückt „↗ Schnell“, bekommt eine Bestätigung, nichts ändert sich	Behoben durch den Haken aus G plus den Hinweis: Bei aktivem Gründlich soll die Bestätigung des Tiefen-Knopfes den Satz tragen „solange Gründlich an ist, läuft alles auf Max“
Dauerbetrieb kostet mehr	Höheres Kontingent, langsamere Antworten	Der sichtbare Haken — das ist genau Adams Anliegen

6. Abnahme (in dieser Reihenfolge)

1. `.venv/bin/python scripts/check_hilfe_buttons.py` ⇒ Exit 0.
2. Selbsttest-Reihe laufen lassen ⇒ alle Punkte grün, insbesondere , Tastatur-Vollständigkeit und der neue Prüfer aus J.3.
3. **Am lebenden Bot:** Knopf drücken ⇒ Haken erscheint, Bestätigung „ist an“. /
`status` zeigt den Modus. Bot neu starten ⇒ **Haken ist immer noch da.**
4. **Gesprächsfaden prüfen** (der stille Fall): bei aktivem Gründlich zwei Nachrichten hintereinander schicken, die zweite mit Bezug auf die erste („und was war das zweite davon?“). Wird der Bezug verstanden, ist D korrekt umgesetzt.

5. Knopf erneut drücken ⇒ Haken weg, Bestätigung „ist aus“, nächste Antwort wieder in normalem Tempo.
-

7. Rückweg

Ein einzelner Commit, der auf einmal zurückgenommen werden kann. Zusätzlich sollte `_USER_PREFS` vor dem ersten Start gesichert werden — der neue Schlüssel `thorough` ist zwar additiv, aber eine Sicherung kostet nichts.

8. Was bewusst NICHT geändert wird

- Der Textzusatz `_THOROUGH_PREFIX` — inhaltlich unverändert richtig.
 - Die Nachrichten-Persistenz (`bot.py:5628-5640`) — die Job-Eigenschaft bleibt auftragsbezogen.
 - Kein Auto-Abschalten nach einer bestimmten Zeit. Adam will den Dauerbetrieb ausdrücklich; eine eingebaute Automatik wäre genau der Zustand, den er gerade abgeschafft haben will.
-

9. Eine Entscheidung, die Adam gehört

Soll Gründlich die Denktiefe weiterhin erzwingen? Heute setzt der Modus die Tiefe fest auf Max. Im Dauerbetrieb heißt das: Der Tiefen-Knopf ist wirkungslos, solange Gründlich an ist.

- **Variante A (in diesem Auftrag gebaut):** Gründlich erzwingt Max. Entspricht dem heutigen Verhalten, ist die höchste Qualität, kostet am meisten. Der Haken auf „ Max“ zeigt es ehrlich an.
- **Variante B:** Gründlich erzwingt nur den Quellencheck, die Tiefe wählt Adam weiter selbst. Günstiger im Dauerbetrieb, aber eine stille Änderung dessen, was „gründlich“ bisher bedeutet hat.

Meine Empfehlung: A — weil der Modus dann bleibt, was er war, und weil der sichtbare Haken das Kostenproblem bereits löst. B wäre ein Thema für später, falls sich der Dauerbetrieb als zu teuer erweist.